



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py`.
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS**: Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS**: Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS**: Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task**: Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State space is the set of all possible complete configurations (states) the environment can be in, where each state holds everything that matters to the task. A search tree is the tree of partial paths the agent builds starting from the initial state by applying actions, where nodes are states and edges are actions. The state space represents all possible situations in the problem, while the search tree is one possible way the agent explores from its current situation toward a goal.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached/explored set records every state already expanded so that no state is expanded twice. It converts a Tree Search (which can revisit the same state forever and loop) into a Graph Search. Without it, my DFS would dive down a corridor, backtrack, and dive into the same corridor again and again; the reached set guarantees each state is visited at most once and prevents infinite loops.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands nodes in order of depth, so the first time it reaches the goal it has used the fewest number of actions. On a grid with equal step costs it is guaranteed to find the shortest (optimal) path. DFS instead follows one deep branch as far as it can before backtracking, so it may hit the goal quickly but along a long, winding route that is not the shortest. Because DFS never compares path costs across different branches, there is no reason its first solution should be best.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space

complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

Memory is the real problem on a huge grid. BFS stores every node on the frontier, so its memory is $O(b^d)$ with branching factor b and goal depth d , which for a 4-direction grid is roughly 4^d - far too much for depths d in the thousands of a 1000x1000 grid. DFS only keeps one path from root to leaf in its stack, so its memory is $O(b \cdot d)$, which stays small even on a very large grid. UCS is between them and can be as bad as BFS in memory because it stores every generated node in the priority queue. So BFS and UCS run out of memory fastest, while DFS trades optimality for a low, predictable memory use.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING